B.Tech CSE Project

Library Management System

in Java using SQLite

Ankita Sarkar
B.Tech – Computer Science & Engineering (2nd Year)
Course: OOPs Lab Project

Introduction

What it is:	A desktop application to manage library books and students using Java.
Core Tech Stack:	Java (OOP) · Java Swing (GUI) · SQLite (Database via JDBC)
Purpose:	Automate book issuing, returns, and record keeping — replacing manual registers.
Data Persistence:	Uses a local SQLite file (library.db) so data is saved between sessions.
Scope:	Designed for a small institutional library — lightweight and self-contained.

Problem Statement

1. Manual library management is time-consuming and error-prone.
2. No easy way to track which student has which book issued.
3. Physical registers are difficult to search and update.
4. Data can be lost due to damage or mismanagement of records.
5. No system to check book availability in real time.

Solution: Build a Java-based desktop system with a database backend for efficient, persistent, and user-friendly library management.

Objectives

Develop a GUI Application

Build an interactive desktop interface using Java Swing.

Implement Core Operations

Support Add Book, Display Books, Issue Book, Return Book.

Apply OOP Principles

Use encapsulation, inheritance, abstraction, and polymorphism.

Integrate SQLite Database

Store all records persistently in a local .db file via JDBC.

Modular Code Design

Organise code into packages: database, models, services, ui.

Ensure Reliability

Handle errors gracefully and keep data consistent across sessions.

Technologies Used

Technology	Category	Key Details
Java (JDK 17+)	Core Language	OOP, Exception Handling, Collections, Threads
Java Swing	GUI Framework	JFrame, JTable, JButton, JTextField, JScrollPane
SQLite	Database	Lightweight, file-based relational DB — library.db
JDBC	DB Connectivity	Java Database Connectivity API to execute SQL queries
SQLite JDBC Driver	Driver	org.sqlite.JDBC — connects Java app to SQLite file

System Architecture (Package Structure)

database	DBConnection, DBSetup Manages SQLite connection and creates tables on startup.
models	Book, Student Plain Java objects (POJOs) representing data entities.
services	LibraryService Business logic — add, issue, return, display books.
ui	LibraryUI Java Swing GUI — buttons, tables, input forms.
Main	Main.java Entry point — initialises DB and launches the UI.

OOP Concepts Used

Encapsulation *Where: Book, Student classes*	Private fields with public getters/setters for data hiding.
Abstraction *Where: LibraryService*	UI calls service methods without knowing SQL internals.
Inheritance *Where: LibraryUI extends JFrame*	Swing components inherit from JFrame for window behaviour.
Polymorphism *Where: Method overloading*	Multiple constructors and overloaded methods in model classes.
Exception Handling *Where: JDBC calls*	Try-catch blocks around all DB operations to handle SQL errors.

Database Design (SQLite — library.db)

Table: books

id	INTEGER	Primary Key (auto)
title	TEXT	Book title
author	TEXT	Author name
is_issued	INTEGER	0 = Available, 1 = Issued

Table: students

id	INTEGER	Primary Key (auto)
name	TEXT	Student name
book_id	INTEGER	Foreign key → books.id

Relationship: students.book_id references books.id (Issue = link student ↔ book)

UML Class Diagram Overview

Book

- id: int
- title: String
- author: String
- isIssued: boolean

Student

- id: int
- name: String
- bookId: int

DBConnection

+ getConnection(): Connection

LibraryService

+ addBook()
+ issueBook()
+ returnBook()
+ getAllBooks()

LibraryUI

+ JTable, JButton, JTextField
- service: LibraryService
+ initUI()

LibraryUI uses LibraryService which uses DBConnection to query Book / Student tables

Features of the System

01

Add Book

Enter title and author to add a new book to the database instantly.

02

Display Books

View all books in a JTable with title, author, and availability status.

03

Issue Book

Assign a book to a student — updates is_issued flag and creates student record.

04

Return Book

Return a book — clears student record and marks book as available.

05

Persistent Storage

All data saved in SQLite (library.db) — persists across application restarts.

06

Error Handling

Graceful error messages for invalid input, missing books, or duplicate issues.

Results / System Output

1

Launch App

Main.java runs → DBSetup creates tables if not exists → LibraryUI window opens.

2

Add a Book

User enters title + author → clicks Add Book → LibraryService inserts into books table.

3

Display Books

Click Display → JTable loads all books from SQLite — shows title, author, status.

4

Issue Book

Enter book ID + student name → book marked is_issued=1 → student record created.

5

Return Book

Enter book ID → student record deleted → book marked is_issued=0 → available again.

Conclusion & Future Scope

✓ Conclusion

- Successfully built a working Library Management System using Java OOP.
- Demonstrated all four OOP principles in a real-world application.
- SQLite via JDBC provides reliable, file-based persistent storage.
- Java Swing GUI makes the system accessible without web or server setup.
- Modular package structure makes code clean and maintainable.

🚀 Future Scope

- Search and filter books by title or author.
- Add login/authentication for admin and students.
- Generate due-date and fine calculation for late returns.
- Export reports to PDF or Excel.
- Migrate to MySQL for multi-user network access.

Thank You!